

nanospice

A SPICE circuit simulator in 1000 lines of Rust

Milan Rother

2026

Abstract

nanospice is a circuit simulator with the classic Berkeley SPICE algorithm set: modified nodal analysis, Newton-Raphson with junction limiting and two operating point fallbacks, sparse LU factorization, trapezoidal transient integration with local truncation error timestep control, and small-signal AC analysis. The entire solver lives in one Rust source file of at most 1000 nonblank, noncomment lines, enforced by a test, with no dependencies outside the standard library. This report walks through the file in order, derives each algorithm, and explains the design decisions the budget forced. The bar under each section heading shows where that code section sits in the line budget.

1 The budget as a design tool

The rule: one file, `src/main.rs`, at most 1000 lines after removing blank lines and comments, standard library only. A test reads the file, counts, and fails the build above the limit. Tests, example netlists, and comments are free. The budget forces every feature to displace another, which makes the cost of each decision explicit, and it keeps the simulator readable in one sitting. That is the purpose of the repository.

This report follows the source file from top to bottom. Every section of the report corresponds to a section of the file, marked by the same name the code uses in its section comments, and carries a bar showing which slice of the budget it occupies. Table 1 is the overview.

code section	lines	cumulative
constants, error and output plumbing	23	23
numbers: <code>Num</code> trait, complex, unit suffixes	78	101
circuit data model	63	164
parser	262	426
sparse LU and stamp helpers	105	531
device models	50	581
solver: assembly, Newton, operating point	162	743
output selection	31	774
analyses and main	217	991

Table 1: Line budget by code section, in file order.

The parser is the largest section: the numerics of a circuit simulator are compact, the input format is not. Everything the budget excluded is listed with reasons in section 13.

2 Background: modified nodal analysis

theory only

0 lines

Everything downstream assembles one linear system per Newton iteration, so the formulation comes first. Plain nodal analysis writes Kirchhoff's current law at every node except ground: the sum of currents leaving each node is zero. A conductance g between nodes p and n carries the current $g(v_p - v_n)$, and collecting terms produces the node conductance matrix. The formulation breaks on ideal voltage sources: a voltage source has no admittance description, its current is whatever the circuit demands.

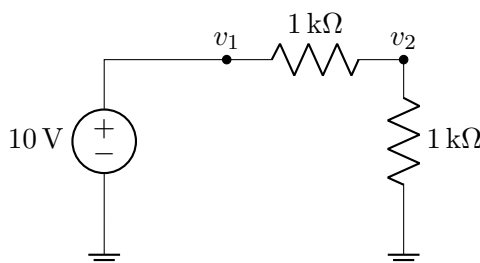
Modified nodal analysis (MNA) repairs this by promoting that current to an unknown. The vector of unknowns becomes

$$x = (v_1, \dots, v_N, i_1, \dots, i_B)^T,$$

with one branch current for every voltage-defined element: independent voltage sources, inductors (which are voltage-defined in DC, where they are shorts), and voltage-controlled voltage sources. Each such element contributes its current $\pm i_b$ to the KCL rows of its two terminal nodes and adds one constraint row of the form $v_p - v_n = E$.

A concrete example, the resistive divider used in the first integration test: a 10 V source into two 1 k Ω resistors, $g = 10^{-3}$ S,

$$\begin{pmatrix} g & -g & 1 \\ -g & 2g & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ i \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 10 \end{pmatrix} \implies v_1 = 10, \quad v_2 = 5, \quad i = -5 \text{ mA}.$$



Two properties matter later. First, the third diagonal entry is a structural zero: any LU factorization of an MNA matrix needs row pivoting. Second, i comes out negative because the branch current is defined flowing into the positive terminal, the SPICE convention: a supply that delivers power reads negative.

Nonlinear devices fit the same frame through linearization, covered in section 9; reactive devices fit through companion models, also there. The result is that one matrix assembly routine serves DC, transient, and AC analysis.

3 Constants and plumbing

constants, error and output plumbing

23 lines, budget 0 to 23

The file opens with the numerical constants, all of them SPICE2 heritage:

constant	value	meaning
GMIN	10^{-12} S	conductance in parallel with every junction
RELTOL	10^{-3}	relative convergence tolerance
VNTOL	$1\ \mu\text{V}$	absolute floor for voltage convergence
ABSTOL	$1\ \text{pA}$	absolute floor for current convergence
TRTOL	7	slack factor on the truncation error estimate
VT	$25.85\ \text{mV}$	thermal voltage kT/q at 27°C
ITL_OP	100	Newton iteration limit, operating point
ITL_TRAN	10	Newton iteration limit per transient step

The two-part tolerance $\text{tol} = \tau + \text{RELTOL} \cdot |x|$ appears in both the Newton test and the timestep control. The relative part scales with signal amplitude; the absolute floor τ (VNTOL for voltages, ABSTOL for currents) keeps nodes that sit near zero from demanding impossible precision. TRTOL compensates for the conservative truncation error estimate; the value 7 is the SPICE2 default.

The remaining plumbing is a `die` helper that prints to `stderr` and exits, and an `out!` macro for `stdout` that exits quietly when the pipe closes. The macro exists because of a Unix detail: Rust ignores `SIGPIPE`, so a plain `println!` panics with a backtrace when the output is piped into `head` and the pipe closes.

4 Numbers

numbers

78 lines, budget 23 to 101

AC analysis needs complex arithmetic and the standard library does not provide it. Rather than duplicating the linear solver for `f64` and a complex type, everything numeric is generic over a three-method trait:

```
trait Num:
  Copy
  + std::ops::Add<Output = Self>
  + std::ops::Sub<Output = Self>
  + std::ops::Mul<Output = Self>
  + std::ops::Div<Output = Self>
{
  fn zero() -> Self;
  fn one() -> Self;
  fn mag(self) -> f64;
}
```

`mag` is the pivoting magnitude: absolute value for reals, modulus for complex. The complex type `Cx` implements the four operator traits in the obvious way; division uses the textbook formula

$$\frac{a + jb}{c + jd} = \frac{(ac + bd) + j(bc - ad)}{c^2 + d^2},$$

which is numerically adequate here because AC matrices of moderate size do not produce the extreme exponent spreads that would call for Smith's algorithm.

Design decision: one generic solver instead of two copies

The trait plus the operator implementations cost about 45 lines. Two copies of the sparse factorization would cost about 55 and could drift apart silently. The generic version also expresses the pivoting strategy once, through `mag`, instead of duplicating it.

The section closes with the SPICE number grammar: the longest prefix of a token that parses as a float, followed by a unit suffix from `{t, g, meg, k, m, u, n, p, f}`. The implementation shortens the token until `str::parse` succeeds, then looks the remainder up in a table ordered so that `meg` matches before `m`:

```
fn num(tok: &str) -> Option<f64> {
    let t = tok.trim();
    if !t.is_ascii() {
        return None;
    }
    let mut end = t.len();
    while end > 0 && t[..end].parse::<f64>().is_err() {
        end -= 1;
    }
    if end == 0 {
        return None;
    }
    let v: f64 = t[..end].parse().unwrap();
    let mult = SUFFIXES.iter().find(|(s, _)| t[end..].starts_with(s)).map_or(1.0,
        |(_, m)| *m);
    Some(v * mult)
}
```

The ASCII guard is a bug fix with a fuzz test behind it: Rust string slicing is byte-indexed and panics inside a multibyte character, so a netlist containing a literal micro sign must produce *bad number*, not a crash.

5 The circuit data model

circuit data model	63 lines, budget 101 to 164

A circuit is four vectors: devices, their names, node names, and the requested analyses, plus the branch count. Devices are one enum:

```
enum Dev {
    R { p: usize, n: usize, v: f64 },
    C { p: usize, n: usize, v: f64, ic: f64 },
    L { p: usize, n: usize, v: f64, br: usize, ic: f64 },
    V { p: usize, n: usize, dc: f64, ac: f64, wave: Option<Wave>, br: usize },
    I { p: usize, n: usize, dc: f64, ac: f64, wave: Option<Wave> },
    D { p: usize, n: usize, is: f64, nf: f64 },
    M { d: usize, g: usize, s: usize, kp: f64, vt0: f64, lambda: f64, pmos: bool },
    E { p: usize, n: usize, cp: usize, cn: usize, k: f64, br: usize },
    G { p: usize, n: usize, cp: usize, cn: usize, k: f64 },
}
```

```
Q { c: usize, b: usize, e: usize, is: f64, bf: f64, br: f64, pnp: bool },
}
```

Design decision: an enum, not a device trait

The alternative is a `Device` trait with a `stamp` method per implementation, which costs a trait definition, one `impl` block per device, and dispatch plumbing. The enum costs one declaration and a `match` arm per device at each use site, and the compiler checks exhaustively that no device is missing in any of them: assembly, output naming, breakpoint collection, state initialization. Three such omissions were caught at compile time while features were added. The device set is closed, so the extensibility a trait would buy is not needed.

Source waveforms are a second enum with the standard SPICE semantics. The damped sine, for $t \geq t_d$:

$$v(t) = v_o + v_a e^{-\theta(t-t_d)} \sin(2\pi f(t-t_d)),$$

and the trapezoidal pulse, piecewise linear through delay, rise, on, fall, off, optionally periodic. Initial conditions on C and L use `f64::NAN` as the *not specified* sentinel, which saves an `Option` wrapper and its pattern matching at every use site; NaN is unambiguous here because a NaN initial condition is meaningless anyway.

6 The parser

parser

262 lines, budget 164 to 426



The parser is the largest section. Preprocessing runs once per line: strip trailing comments after `;`, lowercase, replace parentheses and commas by spaces, then apply the line discipline. The first line of the file is a title, `*` starts a comment line, and `+` splices a continuation onto the previous line. Replacing the parentheses early means `sin(0 1 1k)` and `sin 0 1 1k` tokenize identically, and no grammar for balanced parentheses exists anywhere in the program.

Model cards are collected in a prepass over all lines so that `.model` may appear anywhere in the deck, before or after the devices that reference it. When a device line references a model, the model's parameter tokens are spliced *behind* the inline tokens of the line. Parameter lookup returns the first match, so instance parameters override model parameters by construction, with no explicit precedence logic:

```
let expand = |from: usize| -> Vec<&str> {
  let (mut inline, mut modt) = (Vec::new(), Vec::new());
  for t in toks.iter().skip(from) {
    match models.get(*t) {
      Some(mt) => modt.extend(mt.iter().map(|s| s.as_str())),
      None => inline.push(*t),
    }
  }
  inline.extend(modt);
  inline
};
```

Source specifications (`dc`, `ac`, `sin`, `pulse`) are parsed by a small keyword machine that greedily collects the numeric tokens after each keyword; missing trailing parameters default to zero, matching SPICE. The `.print` card is reconstructed from the parenthesis-stripped token stream: `v out` pairs back into `v(out)`, which works because every output column has exactly the form `kind(name)`.

Errors go through `die` with a named message: unknown device letters, unparseable numbers, missing tokens, unknown sweep sources. A fuzz test feeds seven classes of malformed netlist to the binary and requires a clean named error and no panic for each.

Design decision: no subcircuits

`.subckt` costs roughly a hundred lines of name spacing, port mapping, and flattening. Real decks are hierarchical, so this is the most limiting omission in practice. It was cut in favor of the transistor models.

7 Sparse LU

sparse LU

105 lines, budget 426 to 531



Gaussian elimination factors $PA = LU$ with row pivoting and then solves by substitution. Dense, this is 25 lines and $O(n^3)$, and it was the first implementation; it was replaced once the test suite pinned the behavior. A sparse package with fill-reducing ordering does not fit the budget. The following design, including the stamp helpers, takes 105 lines.

The matrix rows live in the `Sys` struct as vectors of (column, value) pairs kept sorted by column, and the stamp helpers write into them; the ground row and column, index 0, are silently dropped at insertion:

```
fn quad(&mut self, p: usize, n: usize, cp: usize, cn: usize, g: T) {
    self.add(p, cp, g);
    self.add(p, cn, T::zero() - g);
    self.add(n, cp, T::zero() - g);
    self.add(n, cn, g);
}

fn branch(&mut self, p: usize, n: usize, br: usize) {
    self.add(p, br, T::one());
    self.add(n, br, T::zero() - T::one());
    self.add(br, p, T::one());
    self.add(br, n, T::zero() - T::one());
}
```

Design decision: a dummy ground row

Ground is unknown 0 and the solver loops start at 1. The alternative is an *if node is not ground* test inside every stamp, roughly two dozen branches across ten devices. One guard inside `add` replaces all of them, and $x_0 = 0$ is pinned after the solve. The conductance quad above also covers the VCCS (rows at the output nodes, columns at the controlling nodes), and `branch` covers V, L, and E, so the ten devices share two stamp helpers.

The third helper covers every nonlinear device in the file. A static device model is a set of current branches: a current i flowing from node p to node n whose value depends on one or more

controlling node-pair voltages. Linearized at the Newton iterate, such a branch stamps one conductance quad per control and one right-hand side pair. This is the contribution operator of Verilog-A, $I(p, n) <+ f(v)$, as a ten-line method:

```
// Verilog-A style contribution: current i flows p -> n, linearized at
// the (possibly limited) evaluation point and shifted to the raw
// iterate; ctl lists control pairs with d i / d (x[a] - x[b]).
fn contrib(&mut self, p: usize, n: usize, i: T, ctl: &[(usize, usize, T)], x: &[T]) {
    let mut ieq = i;
    for &(a, b, g) in ctl {
        self.quad(p, n, a, b, g);
        ieq = ieq - g * (x[a] - x[b]);
    }
    self.rhs(p, T::zero() - ieq);
    self.rhs(n, ieq);
}
```

Design decision: devices as contributions

The primitive replaced hand-written stamps: the diode is one contribution, the MOSFET one contribution with two controls, the BJT three contributions. The refactor removed 27 lines of stamp code for 10 lines of primitive, and the freed budget paid for the JFET (section 8). Hand-derived Jacobian stamps, like the 3×3 BJT matrix in section 9, become a verification artifact instead of code.

Derivation: the first-entry invariant

The factorization maintains: *before elimination step k , every row stored at index $i \geq k$ contains only columns $\geq k$.*

Base case: column 0 is never stored, so before step 1 all rows hold columns ≥ 1 . Step: at step k the rows at indices $> k$ that contain column k have it eliminated, and the elimination removes that entry from the row; rows without column k are untouched. Afterwards every row at index $> k$ holds only columns $\geq k + 1$. $\square$

Consequence: if a row at index $i \geq k$ has an entry in column k at all, that entry is the *first* element of its sorted vector. Pivot search over column k reduces to comparing the first elements of the candidate rows, an $O(1)$ probe per row instead of a search, with no column-linked index structure to maintain.

The row update against the pivot row is a merge of two sorted sequences. Fill-in, the entries that appear where the original matrix had none, falls out of the merge automatically as the union of the two column sets:

```
// row a minus f times pivot row p, both sorted
fn merge<T: Num>(a: &[(usize, T)], p: &[(usize, T)], f: T) -> Vec<(usize, T)> {
    let mut o = Vec::with_capacity(a.len() + p.len());
    let (mut i, mut j) = (0, 0);
    while i < a.len() && j < p.len() {
        if a[i].0 == p[j].0 {
            o.push((a[i].0, a[i].1 - f * p[j].1));
            i += 1;
            j += 1;
        } else if a[i].0 < p[j].0 {
            o.push(a[i]);
        }
    }
    o.extend(p[j..]);
}
```

```

        i += 1;
    } else {
        o.push((p[j].0, T::zero() - f * p[j].1));
        j += 1;
    }
}
o.extend_from_slice(&a[i..]);
for &(c, v) in &p[j..] {
    o.push((c, T::zero() - f * v));
}
o
}

```

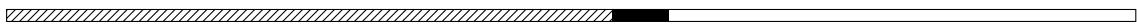
Pivoting selects the largest **mag** among the candidate first entries and swaps rows whole. MNA makes pivoting mandatory, not optional: the constraint row of every voltage source has a structural zero on the diagonal, visible in the divider example of section 2, so a factorization without row exchanges fails on essentially every netlist. If no candidate entry exists for column k , the trailing submatrix has an empty column and is singular; the factorization returns the failing column index and the caller maps it to a node or branch name. The most common user error, a node with no DC path to ground, produces *singular matrix at node b, no conduction path?* instead of a generic failure.

The cost of an elimination step is the sum of the lengths of the merged rows, so the total factorization cost is proportional to the fill-in it creates. For ladder and tree topologies, elimination in netlist order creates none: a 30-stage randomized ladder factors in linear time, and a 2000-node RC ladder solves its operating point plus an AC sweep in tens of milliseconds. The deliberate omission is Markowitz ordering; the worst case therefore degrades toward dense cost, which for the circuit sizes a 1000-line simulator will realistically see is the right trade.

8 Device models

device models

50 lines, budget 531 to 581



This section holds the semiconductor mathematics: the diode evaluation, the junction limiting shared by diode and BJT, and the MOSFET square law.

Diode. Shockley equation with emission coefficient n and the global g_{min} in parallel:

$$i = I_S(e^{v/nV_T} - 1) + g_{min}v, \quad g_d = \frac{I_S}{nV_T}e^{v/nV_T} + g_{min}.$$

The exponent is clamped at 200 as an overflow backstop; in normal operation the limiter below keeps it far smaller.

Derivation: the critical voltage of pnjlim

Newton diverges on the bare exponential: an early iterate proposing $v = 5\text{ V}$ puts $e^{v/V_T} \approx e^{193}$ into the Jacobian. The classic limiter damps the proposed junction voltage once it passes the critical voltage v_{crit} , defined as the point of maximum curvature of the $i(v)$

characteristic. With $i' = (I_S/V_T)e^{v/V_T} =: u$ and $i'' = u/V_T$, the curvature is

$$\kappa(v) = \frac{i''}{(1+i'^2)^{3/2}} = \frac{u}{V_T(1+u^2)^{3/2}}, \quad \frac{d\kappa}{du} = \frac{1-2u^2}{V_T(1+u^2)^{5/2}} = 0 \implies u = \frac{1}{\sqrt{2}}.$$

Since u grows monotonically in v , the maximum is unique, and substituting back:

$$\frac{I_S}{V_T}e^{v/V_T} = \frac{1}{\sqrt{2}} \implies v_{crit} = V_T \ln \frac{V_T}{\sqrt{2} I_S}.$$

For $I_S = 10^{-14}$ A this is about 0.72 V. Beyond v_{crit} , **pnjlim** replaces a proposed step by a logarithmic one, $v \leftarrow v_{old} + V_T \ln(1 + (v_{new} - v_{old})/V_T)$, which advances the junction a controlled amount per iteration regardless of how far Newton overshoots.

```
fn pnjlim(vnew: f64, vold: f64, vt: f64, vcrit: f64) -> f64 {
  if vnew > vcrit && (vnew - vold).abs() > 2.0 * vt {
    if vold > 0.0 {
      let arg = 1.0 + (vnew - vold) / vt;
      if arg > 0.0 { vold + vt * arg.ln() } else { vcrit }
    } else {
      vt * (vnew / vt).ln()
    }
  } else {
    vnew
  }
}
```

A wrapper, **junction**, adds the bookkeeping every junction needs: computing v_{crit} , remembering the last limited voltage per device, and reporting whether limiting actually changed the proposal, which the Newton loop consumes (section 9). The diode uses one junction, the BJT two, which is why the limiter memory is two values per device.

MOSFET, level 1. The square law with channel-length modulation, $v_{ov} = v_{GS} - V_{T0}$:

$$i_D = \begin{cases} 0 & v_{ov} \leq 0 \\ K_P (v_{ov} v_{DS} - \frac{1}{2} v_{DS}^2) (1 + \lambda v_{DS}) & 0 < v_{DS} < v_{ov} \\ \frac{1}{2} K_P v_{ov}^2 (1 + \lambda v_{DS}) & v_{DS} \geq v_{ov} \end{cases}$$

with the small-signal parameters obtained by differentiation, $g_m = \partial i_D / \partial v_{GS}$ and $g_{ds} = \partial i_D / \partial v_{DS}$; in saturation $g_m = K_P v_{ov} (1 + \lambda v_{DS})$ and $g_{ds} = \frac{1}{2} K_P v_{ov}^2 \lambda$, so a nonzero λ is what gives the transistor a finite output resistance.

Derivation: the effective frame and the vanishing signs

The model must serve four cases: nmos and pmos, each with either terminal acting as the source. Define the polarity $s = +1$ for nmos, -1 for pmos, and choose effective terminals (d_e, s_e) so that $v_{DS}^e = s(v_{d_e} - v_{s_e}) \geq 0$; evaluate the square law in this frame. The physical current into d_e is $I = s i_D$. Now take any derivative, for example with respect to the gate voltage:

$$\frac{\partial I}{\partial v_g} = s \cdot \frac{\partial i_D}{\partial v_{GS}^e} \cdot \frac{\partial v_{GS}^e}{\partial v_g} = s \cdot g_m \cdot s = g_m.$$

Every derivative picks up the polarity twice, once from the current and once from the voltage transformation, and $s^2 = 1$. The conductance stamps are therefore identical for all four cases; only the right-hand side term carries s . One sign variable and a conditional swap replace a four-way case split:

```
fn mos_op(dv: &Dev, x: &[f64]) -> (usize, usize, f64, f64, f64) {
    let Dev::M { d, g, s, kp, vt0, lambda, pmos } = dv else { unreachable!() };
    let sg = if *pmos { -1.0 } else { 1.0 };
    let (ed, es) = if sg * (x[*d] - x[*s]) >= 0.0 { (*d, *s) } else { (*s, *d) };
    let vgs = sg * (x[*g] - x[es]);
    let vds = sg * (x[ed] - x[es]);
    let (id, gm, gds) = mos_eval(vgs, vds, *kp, *vt0, *lambda);
    (ed, es, sg * id, gm, gds)
}
```

JFET. The level 1 JFET is the square law again: saturation current $\beta(v_{GS} - V_{TO})^2$ with a depletion threshold $V_{TO} < 0$, and the triode region matches the MOSFET equations with $K_P = 2\beta$. The parser maps the J device onto the MOSFET variant with $K_P = 2\beta$ and a default $V_{TO} = -2\text{ V}$; evaluation, stamps, and AC analysis come along unchanged, and the gate junction is not modeled. The device shares the MOSFET parser arm, so it costs a conditional in the parameter defaults and nothing anywhere else.

9 The solver

solver: assembly, Newton, operating point 162 lines, budget 581 to 743



Companion linearization. A nonlinear device current $i(v)$ enters the linear system as its tangent at the current Newton iterate $v^{(k)}$:

$$i \approx g v + I_{eq}, \quad g = \left. \frac{\partial i}{\partial v} \right|_{v^{(k)}}, \quad I_{eq} = i(v^{(k)}) - g v^{(k)},$$

a conductance in parallel with a current source. Assembling and solving the resulting linear system is exactly one Newton-Raphson iteration on the full nonlinear network equations; the derivation is the multivariate Taylor expansion truncated after the linear term, applied device by device.

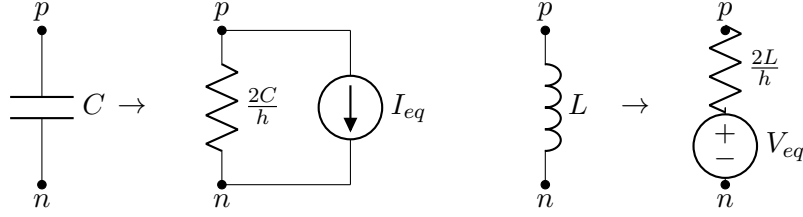
Derivation: trapezoidal companion models

In transient analysis the reactive devices become resistive companions. The trapezoidal rule applied to $i = C dv/dt$ over a step h :

$$\frac{i_{n+1} + i_n}{2} = C \frac{v_{n+1} - v_n}{h} \implies i_{n+1} = \underbrace{\frac{2C}{h}}_{g_{eq}} v_{n+1} - \left(\frac{2C}{h} v_n + i_n \right),$$

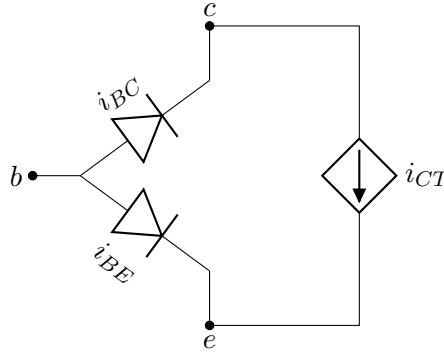
a conductance g_{eq} with a history current source: the capacitor is a resistor for the duration of one step. The inductor is the dual, $v_{n+1} = (2L/h)(i_{n+1} - i_n) - v_n$, stamped into its

branch row since its current is already an MNA unknown. Each reactive device keeps a two-entry state, its value and the dual quantity, updated after every accepted step.



Backward Euler would give $g_{eq} = C/h$ with a simpler history term and first-order accuracy; the trapezoidal rule is second order and A-stable with no numerical damping, which is why an LC tank simulated for twenty periods keeps its amplitude (a test enforces this). The price of zero damping is ringing at discontinuities, addressed by the breakpoint handling in section 11.

The BJT. The Ebers-Moll model in transport form is three current branches: the two base junctions and the transport source from collector to emitter.



With $e_f = e^{v_{BE}/V_T}$, $e_r = e^{v_{BC}/V_T}$:

$$i_{CT} = I_S(e_f - e_r), \quad i_{BE} = \frac{I_S}{\beta_F}(e_f - 1), \quad i_{BC} = \frac{I_S}{\beta_R}(e_r - 1),$$

$$I_C = i_{CT} - i_{BC}, \quad I_B = i_{BE} + i_{BC}, \quad I_E = -(I_C + I_B).$$

The transport form references one saturation current for the main transfer, which keeps forward and reverse operation consistent through a single i_{CT} . In the assembly, each branch is one contribution: the junctions with their own conductance, the transport source controlled by both junction voltages,

```
s.contrib(*b, *e, sg * (ibe + gpi * dbe), &[*b, *e, gpi], x);
s.contrib(*b, *c, sg * (ibc + gmu * dbc), &[*b, *c, gmu], x);
s.contrib(*c, *e, sg * (ict + gmf * dbe - gmr * dbc),
    &[*b, *e, gmf], (*b, *c, -gmr), x);
```

where `dbe` and `dbc` shift the linearization from the limited junction voltages back to the raw iterate. Expanding the three contributions by hand gives the classic 3×3 Jacobian stamp over

(v_b, v_c, v_e) , with $g_{mf} = \partial i_{CT} / \partial v_{BE}$, $g_{mr} = -\partial i_{CT} / \partial v_{BC}$, $g_\pi = \partial i_{BE} / \partial v_{BE}$, $g_\mu = \partial i_{BC} / \partial v_{BC}$:

$$\begin{pmatrix} g_{mf} - g_{mr} - g_\mu & g_{mr} + g_\mu & -g_{mf} \\ g_\pi + g_\mu & -g_\mu & -g_\pi \\ g_{mr} - g_{mf} - g_\pi & -g_{mr} & g_{mf} + g_\pi \end{pmatrix}$$

for the rows I_C, I_B, I_E . Rows and columns each sum to zero: shifting all node voltages together must change nothing, and KCL must close. In the contribution form this holds by construction, per branch. The pnp reuses the npn equations through the same sign argument as the MOSFET; both junctions go through `junction` with their own limiter memory. An integration test builds a common-emitter stage as npn and as pnp mirror image and requires exact symmetry of the solutions.

Newton iteration and convergence. Convergence is declared when every unknown moves less than its two-part tolerance between iterates, and at least two iterations have run so a fresh linearization has confirmed the point:

```
fn newton(ckt: &Circuit, x: &mut Vec<f64>, lim: &mut [[f64; 2]], mode: &Mode, gmin
: f64, maxit: usize) -> Option<usize> {
  let nn = ckt.nodes.len();
  for it in 1..=maxit {
    let mut limited = false;
    let s = assemble(ckt, x, lim, mode, gmin, &mut limited);
    let xn =
      solve(s).unwrap_or_else(|k| die(&format!("singular matrix at {}", no
conduction path?", unk_name(ckt, k))));
    let conv = (1..xn.len()).all(|i| (xn[i] - x[i]).abs() <= tolv(i, nn, xn[i]
], x[i]));
    *x = xn;
    if conv && !limited && it > 1 {
      return Some(it);
    }
  }
  None
}
```

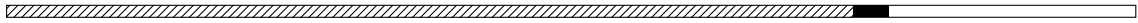
Field note: limiting must deny convergence

The `limited` flag records a bug found during development. With limiting alone, the following happened on a common-emitter amplifier: the bias divider put the base at 2.1 V while the emitter sat at ground, `pnjlim` clamped the base-emitter voltage seen by the stamps to 0.11 V, the transistor therefore conducted nothing, the node voltages stopped changing, and the delta-based convergence test declared success with the transistor off. The output was superficially plausible: the divider voltage was correct, but unloaded. The repair is the classic one: an iteration in which a limiter changed a junction voltage is not allowed to converge. With the flag in place, the same circuit walks the junction up in logarithmic steps and reaches the correct bias, base at 2.05 V and 1.3 mA of collector current, in a dozen iterations. A convergence test on the solution vector alone cannot detect active limiting; the flag makes it visible.

Continuation fallbacks. When plain Newton fails, the operating point is found by continuation: replace the hard problem $F(x) = 0$ by a family $H(x, \lambda) = 0$ that is easy at $\lambda = 0$, coincides with F at $\lambda = 1$, and is tracked by warm-started Newton solves along the path. Two

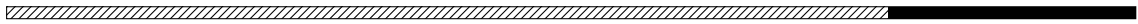
standard families run in order. Gmin stepping puts a conductance g across every junction and walks it from 10^{-2} S down to 10^{-12} S decade by decade: at large g the circuit is nearly linear and Newton converges reliably. Source stepping scales every source value by λ ramped from 0.1 to 1.0: the zero-source circuit has the trivial solution $x = 0$, and each increment moves the solution a small, trackable distance. Gmin stepping handles floating-junction trouble; source stepping handles feedback and bistable topologies, where gmin stepping can converge to the wrong branch. Together they are the standard SPICE fallback chain, and a cross-coupled NMOS latch in the test suite exercises the path.

10 Output selection

output	31 lines, budget 743 to 774
	

Output columns are the node voltages in netlist order followed by the branch currents in device order, named `v(node)` and `i(device)`. A `.print` card filters this list by name; unknown names warn on stderr rather than fail, so a typo drops a column instead of aborting the run. The selection maps names to unknown indices once and every analysis row is then a gather over that index list.

11 Analyses

analyses and main	217 lines, budget 774 to 991
	

Operating point and DC sweep. `.op` is one call into the operating point solver. `.dc` sweeps a source value and re-solves at each point, warm-starting from the previous solution, a continuation along the sweep variable: on a smooth branch each solve takes two or three iterations. The sweep replaces the source specification; any transient waveform on the swept source is dropped.

Transient timestep control. The integrator adapts its step to the local truncation error (LTE) of the trapezoidal rule.

Derivation: the Milne estimate and the cube-root rule
The trapezoidal rule commits the local error $\varepsilon_{tr} = -\frac{h^3}{12} \ddot{x}(\xi)$ per step. The third derivative is estimated for free with a predictor: a quadratic polynomial through the last three accepted points, extrapolated to the new time. With step history d_1, d_2 and new step h, Lagrange extrapolation gives $x_p = \ell_0 x_n + \ell_1 x_{n-1} + \ell_2 x_{n-2},$ $\ell_0 = \frac{(h+d_1)(h+d_1+d_2)}{d_1(d_1+d_2)}, \quad \ell_1 = -\frac{h(h+d_1+d_2)}{d_1 d_2}, \quad \ell_2 = \frac{h(h+d_1)}{(d_1+d_2)d_2},$ whose interpolation error at the new point is $\varepsilon_p = \frac{1}{6} \ddot{x} h(h+d_1)(h+d_1+d_2)$. Corrector and predictor err against the same true solution with errors proportional to the same $\ddot{x}$,

so their difference measures it:

$$x_c - x_p = \varepsilon_p - \varepsilon_{tr} \implies \varepsilon_{tr} \approx \frac{h^3/12}{h(h+d_1)(h+d_1+d_2)/6 + h^3/12} (x_c - x_p),$$

which for uniform steps ($d_1 = d_2 = h$) reduces to $\varepsilon_{tr} \approx (x_c - x_p)/13$, the classic factor. The error is compared per unknown against $\text{TRTOL} \cdot \text{tol}_i$; with the error scaling as h^3 , the step that would exactly meet the tolerance satisfies $h' = h(1/r)^{1/3}$ where r is the worst error ratio, and the controller applies it with a safety factor of 0.9, clamped to $[0.3, 2] \times h$ and capped at `tstep`. Rejected steps ($r > 1$) are retried smaller.

```
let etrap = hs * hs * hs / 12.0;
let fac = etrap / (hs * (hs + d1) * (hs + d1 + d2) / 6.0 + etrap);
let mut r: f64 = 0.0;
for i in 1..m {
    r = r.max(fac * (xn[i] - xp[i]).abs() / (TRTOL * tolv(i, nn, xn[i], x[i])));
}
let scale = 0.9 / r.max(1e-8).cbrt();
```

The predictor doubles as the Newton starting point, which is free and usually saves an iteration. Before three points of history exist, and after a nonconvergent step (cut by 8, the SPICE2 default), the iteration count controls the step instead. With `uic` the operating point is skipped entirely and the `ic=` values seed the reactive state, which is how an undriven LC tank can start oscillating at all: its operating point is identically zero.

Field note: pulse edges inside a step

The first RC test against the analytic step response failed at exactly the first output point. A pulse edge at $t \approx 0$ inside the first step $[0, h]$ is invisible to the method: the capacitor's history current at the interval start still belongs to the pre-edge circuit, so the trapezoidal update gives

$$v_1 = \frac{h/2\tau}{1 + h/2\tau} \approx 0.048 \quad \text{instead of} \quad 1 - e^{-h/\tau} \approx 0.095$$

for $h/\tau = 0.1$: a factor-two error at the first point, and no smaller h fixes the ratio, because the history term is wrong by construction. The LTE controller cannot see it either, since the predictor has no history yet. The classic remedy is exact rather than adaptive: every corner of every pulse source (delay, rise end, fall start, fall end, per period) is registered as a breakpoint, steps are clamped to land on breakpoints exactly, and after crossing one the integrator restarts with a small step and cleared predictor history, because the waveform derivative is discontinuous there.

Small-signal AC. Write every waveform as a small phasor perturbation around the operating point, $x(t) = x_0 + \text{Re}\{\hat{x}e^{j\omega t}\}$, insert into the network equations, and keep first-order terms:

$$(G + j\omega C) \hat{x} = \hat{b},$$

where G is the Jacobian of the resistive equations at x_0 , C collects the reactances, and $\hat{b}$ holds the `ac` source values. The implementation uses the following property: at a converged operating point the limiters pass voltages through unchanged, so *G is exactly the matrix the last Newton iteration already assembles*. The AC path calls `assemble` once, maps the rows to complex, adds $j\omega C$ quads and the $-j\omega L$ branch entries, and hands the result to the same generic sparse LU.

No device model contains any AC-specific code; adding a device to DC automatically adds it to AC. Frequencies sweep linearly or per decade ($f_k = f_1 \cdot 10^{k/n}$), and each unknown reports magnitude and phase.

12 Validation

tests/cli.rs

271 lines, 0 of 991: tests are free

The test suite is 19 integration tests that run the built binary on real netlists and parse its CSV. Tests do not count toward the budget, so robustness lives there, and the suite prefers references that are independent of the implementation: closed-form solutions, physical invariants, and reductions computed separately in the test itself.

test	reference
op_divider	voltage division, source current sign convention
op_diode	KCL closure against the Shockley equation
op_mos	saturation current $\frac{1}{2}K_P v_{ov}^2$, exact bias
op_jfet	JFET drain current βV_{TO}^2 at $v_{GS} = 0$
op_model_card	$n=2$ diode drop, fails if the model is not resolved
op_bjt	$I_C = \beta_F I_B$ and exact npn/pnp mirror symmetry
op_ladder	201 equal resistors, interior node voltage and current
op_random_ladder	LCG-randomized ladder vs. Thevenin reduction in the test
op_latch	cross-coupled NMOS latch, hard operating point
dc_sweep	linearity of the swept divider
tran_rc, tran_rl	step responses vs. $1 - e^{-t/\tau}$
tran_lte_fast_tau	$\tau = tstep/10$, LTE control must resolve the edge
tran_uic_lc	LC amplitude $i_0 \sqrt{L/C}$ from initial conditions
tran_lc_energy	no numerical damping over twenty periods
ac_rc	$1/\sqrt{2}$ magnitude and -45° at the corner
print_selection	column filtering
clean_errors	seven malformed decks: named errors, no panics
loc_budget	the budget itself

Two tests carry the most weight. The randomized ladder builds a 30-stage resistor network from a seeded LCG, computes every node voltage by backward Thevenin reduction and forward division inside the test, and compares node by node at 10^{-6} relative: this validates the sparse factorization, the pivoting, and the fill-in handling on unstructured values with an algorithm that shares no code with the solver. The energy test runs the LC tank for twenty periods and rejects any numerical damping, which pins the choice of the trapezoidal rule: backward Euler fails this test by design.

13 The cut list

The most consequential physical cut is the junction capacitances: without them the semiconductors are purely static, transistors switch instantaneously, and a ring oscillator has no defined frequency. It would be the first addition if the budget grew.

On language: the device enum with exhaustive matching carries the parser, the assembly, and the output selection, and is the main reason the budget holds. C++ would trade the

feature	why it lost	est. lines
<code>.subckt</code> hierarchy	lost to the transistor models	~100
Markowitz ordering	netlist order suffices at this scale	~80
Gummel-Poon BJT	Ebers-Moll covers the teaching core	~80
junction capacitances	needs charge state on every junction	~60
noise analysis	needs the adjoint system	~80
<code>.param</code> expressions	a calculator is its own project	~60
temperature sweeps	V_T is a constant, by choice	~25
current-based convergence check	limiting denial covers the failure mode	~15

Table 2: Features that did not fit, with the reason and the estimated cost.

roughly 60 lines the Num trait and complex type cost against visitor or inheritance boilerplate around `std::variant`, roughly a wash with fewer static guarantees. Fortran is strong on dense arrays, which this simulator does not use, and weak on the parser and the sparse row structure. Python would roughly halve the line count at a different performance class.

14 Closing

The final count is 991 of 1000 lines, with the classic algorithm set complete: MNA, sparse LU with partial pivoting, a Verilog-A style contribution primitive for the device stamps, Newton with `pnjlim` and both continuation fallbacks, trapezoidal integration with LTE control and breakpoints, small-signal AC, eleven device letters, four analyses, model cards, initial conditions, and output selection. Nine lines of headroom remain.